

Documentação do Processo de Login - Projeto Mestre Barbeiro

Bem-vindo(a) à documentação do processo de login do projeto **Mestre Barbeiro**. Este documento foi criado especialmente para explicar, passo a passo e de forma bem simples, como funciona a etapa de login na nossa aplicação. Se você é um desenvolvedor iniciante, não se preocupe! Vamos passar por cada detalhe, linha por linha, explicando o "porquê" e o "como" as coisas funcionam.

1. Visão Geral do Processo

Quando um usuário tenta entrar no sistema (fazer login), ocorre uma "conversa" entre duas partes principais do projeto:

- **Frontend (A "Cara" do site):** Onde o usuário digita o e-mail e a senha. Feito com **React**, **Next.js** e **TypeScript**.
- **Backend (O "Cérebro" e "Memória"):** Onde verificamos se o usuário existe no banco de dados e se a senha está correta. Feito com **C#** e **.NET**.

O Fluxo (Passo a Passo Rápido)

1. O usuário entra na página de login e digita seu e-mail e senha.
2. O Frontend pega esses dados e envia um "pedido" (uma requisição) para o Backend.
3. O Backend recebe o pedido, procura o e-mail no banco de dados e verifica se a senha bate (usando criptografia para segurança).
4. Se tudo estiver certo, o Backend cria um "crachá de acesso" chamado **Token JWT** e devolve para o Frontend.
5. O Frontend guarda esse "crachá" no navegador e redireciona o usuário para a tela inicial (Dashboard).

2. O Frontend (Next.js, React e TypeScript)

O Frontend é construído usando **React** (uma biblioteca para criar interfaces de usuário), **Next.js** (um framework que facilita a criação de rotas e páginas no React) e **TypeScript** (um JavaScript com superpoderes que ajuda a evitar erros adicionando "tipos" às variáveis).

2.1. A Página de Login (app/login/page.tsx)

No Next.js, as páginas são baseadas em pastas. A pasta `app/login` com o arquivo `page.tsx` indica que quando acessarmos `site.com/login`, este código será executado.

```
"use client";
import LoginPage from "../../components/LoginPage/login";

export default function PageLogin() {
  return (
    <LoginPage/>
  );
};
```

Explicação Linha por Linha:

- `"use client";`: Isso avisa ao Next.js que este código deve rodar no navegador do usuário (cliente), pois ele vai interagir com a tela.
- `import LoginPage...`: Aqui estamos "importando" (trazendo) o código visual da página de login que está em outro arquivo.
- `export default function PageLogin() { ... }`: Criamos uma função principal que representa esta página. O `export default` permite que o Next.js encontre essa página e a mostre na tela.
- `return ( <LoginPage/> );`: A função simplesmente exibe o componente `LoginPage` que importamos. Isso deixa o código mais organizado.

2.2. O Componente Visual do Login (components/LoginPage/login.tsx)

Este é o arquivo onde a mágica visual acontece (os campos de texto, os botões, etc).

```
"use client";
import { FormEvent, useState } from "react";
import { Mail, Lock, Eye, EyeOff } from "lucide-react";
import { LoginRequest } from "@/types/auth";
import { useRouter } from "next/navigation";
import { loginUsuario } from "@/services/api/authService";

export default function LoginForm() {
  const route = useRouter();
  const [showPassword, setShowPassword] = useState(false);
  const [isRequest, setIsRequest] = useState(false);
  const [password, setPassword] = useState("");
  const [email, setEmail] = useState("");
```

Explicação:

- Importamos ferramentas do React (`useState` para guardar os dados que o usuário digita, `FormEvent` para lidar com o envio do formulário).
- Importamos ícones da biblioteca `lucide-react` (como Cadeado, Olho, Carta).
- Importamos o `useRouter` do Next.js, que serve para mudar de página (redirecionar).
- `const route = useRouter();`: Inicializa a ferramenta de redirecionamento.
- `const [email, setEmail] = useState("");`: Aqui criamos uma "memória" para o componente. `email` guarda o valor atual, e `setEmail` é a função usada para atualizar esse valor. O mesmo vale para a senha (`password`) e para mostrar/ocultar a senha (`showPassword`).

A Função que Faz o Login (handleLogin)

Dentro do mesmo arquivo, temos a função que é chamada quando o usuário clica em "ENTRAR".

```

async function handleLogin(event: FormEvent<HTMLFormElement>) {
  event.preventDefault();
  setIsRequest(true);

  if (!email || !password) {
    alert("Email ou senha incorretos");
    return;
  }

  const userLogin: LoginRequest = {
    email: email,
    senha: password,
  };

  const response = await loginUsuario(userLogin);

  try {
    if (!response.sucesso) {
      alert("Erro:" + response.message);
      setIsRequest(false);
      return;
    }

    if (response.token && response.role) {
      localStorage.setItem("token", response.token);
      localStorage.setItem("role", response.role);
      alert(response.message);
      route.push("/dashboard");
    }
  } catch (err) {
    const mensagem = err instanceof Error ? err.message : "Erro ao fazer login.";
    alert(mensagem);
  }
}

```

Explicação Linha por Linha:

- `async function handleLogin(event):` A palavra `async` significa que essa função fará algo que pode demorar (como falar com o backend), então ela vai trabalhar de forma "assíncrona".
- `event.preventDefault();`: Impede que a página recarregue ao clicar no botão (que é o comportamento padrão de formulários em HTML).
- `setIsRequest(true);`: Avisa ao sistema que estamos fazendo um pedido (útil para desabilitar o botão e não deixar o usuário clicar várias vezes).
- `if (!email || !password):` Verifica se o e-mail ou a senha estão vazios. Se sim, mostra um alerta e para o processo (`return`).
- `const userLogin: LoginRequest = { ... }:` Empacotamos o e-mail e senha num "pacote" no formato que o backend espera (`LoginRequest`).
- `const response = await loginUsuario(userLogin);`: Aqui chamamos o serviço que fala com o backend. O `await` faz o código esperar a resposta do backend antes de continuar.
- `if (!response.sucesso):` Se o backend disser que deu erro (senha errada, etc), mostramos um alerta.
- `localStorage.setItem("token", response.token);`: Se deu tudo certo, guardamos o "crachá" (`token`) e a função do usuário (`role`) no `localStorage`, que é um pequeno espaço de memória no navegador do usuário.
- `route.push("/dashboard");`: Redireciona o usuário para a tela principal (Dashboard) após o sucesso!

2.3. O Serviço de Comunicação (services/api/authService.ts)

Este arquivo é o carteiro. Ele pega os dados do frontend e envia para o backend usando a internet.

```

export async function loginUsuario(dados: LoginRequest): Promise<LoginResponse> {
  try {
    const resposta = await fetch(`${API_URL}/api/auth/login`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(dados),
    });

    const result = await resposta.json();

    // ... código de verificação ...

    return {
      sucesso: true,
      message: result.message,
      token: result.token,
      role: result.role,
    };
  } catch (err) { ... }
}

```

Explicação:

- `fetch(...)`: É a função do navegador para fazer pedidos na internet.
- `method: "POST"`: Estamos "enviando" dados novos (postando), não apenas lendo (GET).
- `body: JSON.stringify(dados)`: Transforma nosso "pacotinho" de dados do JavaScript num texto formato JSON, que é a língua universal que o Backend entende.
- `await resposta.json()`: Pega a resposta de texto que veio do backend e converte de volta para um objeto JavaScript.

3. O Backend (C# e .NET)

O Backend é construído com **C#** (uma linguagem forte e segura da Microsoft) e o framework **.NET** (que facilita a criação de servidores web e APIs).

3.1. O Controlador de Autenticação (Controllers/AuthController.cs)

O Controlador (Controller) é a porta de entrada do backend. Ele recebe os pedidos da internet e decide quem vai resolver o problema.

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;

    public AuthController(IAuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("login")]
    public async Task<ActionResult> Login([FromBody] UsuarioLoginDTO usuarioLoginDto)
    {
        var usuarioLogin = await _authService.LoginAsync(usuarioLoginDto);

        if (!usuarioLogin.Sucesso)
        {
            return Unauthorized(new { Sucesso=usuarioLogin.Sucesso, Message = usuarioLogin.Message });
        }

        return Ok(new { Sucesso=usuarioLogin.Sucesso, Message = usuarioLogin.Message, Token = usuarioLogin.Token, Role =
usuarioLogin.Role });
    }
}
```

Explicação:

- `[ApiController]` e `[Route("api/[controller]")]`: Configuram essa classe para ser um ponto de acesso web. O caminho dela será `/api/auth` (porque o nome é `AuthController`).
- `public AuthController(IAuthService authService)`: É o construtor. Aqui o C# "injeta" o serviço de autenticação automaticamente, para não precisarmos criá-lo do zero.
- `[HttpPost("login")]`: Define que a função abaixo vai responder quando alguém acessar `/api/auth/login` enviando dados (POST).
- `Task<ActionResult> Login([FromBody] UsuarioLoginDTO usuarioLoginDto)`: A função que recebe os dados do usuário. O `[FromBody]` diz que os dados vêm no "corpo" da requisição. `UsuarioLoginDTO` (Data Transfer Object) é apenas um molde para receber exatamente o E-mail e a Senha, nada a mais.
- `var usuarioLogin = await _authService.LoginAsync(usuarioLoginDto)`: Passa o trabalho pesado para o "Service" (que veremos a seguir).
- Se não tiver sucesso (`!usuarioLogin.Sucesso`), retorna `Unauthorized` (Erro 401 - Não Autorizado). Se der certo, retorna `Ok` (Sucesso 200) com o Token (o crachá).

3.2. A Lógica de Negócio (Services/AuthService.cs)

É aqui que o trabalho de verdade acontece. Vamos verificar o banco de dados e gerar a segurança.

```
public async Task<(bool Sucesso, string Message, string? Token, string? Role)> LoginAsync(UsuarioLoginDTO usuarioLoginDTO)
{
    // 1. Busca o usuário no banco de dados pelo e-mail
    var encontrarUsuario = await _context.Usuarios.FirstOrDefaultAsync(u => u.Email == usuarioLoginDTO.Email);

    if (encontrarUsuario == null)
    {
        return (false, "E-mail ou senha inválidos", null, null);
    }

    // 2. Verifica se a senha está correta usando criptografia
    bool senhaValida = BCrypt.Net.BCrypt.Verify(usuarioLoginDTO.Senha, encontrarUsuario.Senha);

    if (!senhaValida)
    {
        return (false, "E-mail ou senha inválidos", null, null);
    }

    // 3. Cria o "Crachá" (Token JWT)
    var token = GenerateTokenJwt(encontrarUsuario);

    return (true, "Login realizado com sucesso", token, encontrarUsuario.Role);
}
```

Explicação:

- `await _context.Usuarios.FirstOrDefaultAsync(...)`: O Entity Framework (ferramenta de banco de dados do .NET) vai lá no banco de dados, na tabela de `Usuários`, e procura o primeiro usuário que tenha o e-mail igual ao que foi digitado.
- `BCrypt.Net.BCrypt.Verify(...)`: Nunca salvamos a senha "pura" no banco (ex: "123456"). Ela é criptografada e vira um texto gigante misturado. Essa função pega a senha que o usuário digitou agora, criptografa do mesmo jeito e compara para ver se bate com a que está salva no banco.
- `GenerateTokenJwt`: Se tudo deu certo, geramos o token.

Gerando o Token JWT (O Crachá)

Ainda no `AuthService.cs`:

```
private string GenerateTokenJwt(Usuario usuario)
{
    var jwtKey = _configuracao["Jwt:Key"] ?? "";
    var keyBytes = Encoding.ASCII.GetBytes(jwtKey);
    var chaveSimetrica = new SymmetricSecurityKey(keyBytes);
    var credenciais = new SigningCredentials(chaveSimetrica, SecurityAlgorithms.HmacSha256Signature);

    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, usuario.Id.ToString()),
        new Claim(ClaimTypes.Email, usuario.Email),
        new Claim(ClaimTypes.Name, usuario.Name),
        new Claim(ClaimTypes.Role, usuario.Role)
    };

    var descritorToken = new SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(claims),
        Expires = DateTime.UtcNow.AddHours(2),
        Issuer = _configuracao["Jwt:Issuer"],
        Audience = _configuracao["Jwt:Audience"],
        SigningCredentials = credenciais
    };

    var manipuladorToken = new JwtSecurityTokenHandler();
    var tokenCriado = manipuladorToken.CreateToken(descritorToken);

    return manipuladorToken.WriteToken(tokenCriado);
}
```

Explicação:

- **O que é um JWT?** É um formato de "crachá" digital muito usado na web. Ele guarda informações seguras.
- **jwtKey:** Uma senha secreta super complexa que só o backend sabe. Ela serve para assinar o crachá e garantir que ninguém o falsifique.
- **claims:** São as informações públicas que vão dentro do crachá. Exemplo: Nome do usuário, ID, E-mail e o "Role" (que diz se ele é um cliente comum, barbeiro ou administrador).
- **Expires = DateTime.UtcNow.AddHours(2):** O crachá tem validade! Neste caso, expira em 2 horas. Depois disso, o usuário precisará fazer login novamente. Por segurança, se alguém roubar o crachá, não poderá usá-lo para sempre.
- O código cria e empacota tudo isso e retorna uma "string" (um texto longo) que é o Token propriamente dito.

4. Resumo e Conclusão

1. O Usuário aperta **"Entrar"** no React (Frontend).
2. O React transforma isso num JSON e faz um **POST** para `/api/auth/login`.
3. O C# (Backend) recebe os dados no **AuthController**.
4. O **AuthService** vai no banco de dados, acha o e-mail, verifica a criptografia da senha.
5. Se for válido, o backend gera um **JWT Token** (com 2h de validade).
6. O C# devolve o Token pro React.
7. O React guarda o Token no **localStorage**.
8. O React joga o usuário para a tela do **Dashboard**.

Seja muito bem-vindo à programação! Todos os sistemas modernos funcionam seguindo uma base muito parecida com essa. Entender esse fluxo é o primeiro grande passo para se tornar um desenvolvedor incrível!